

Casting

Casting kavramı ile C de karşılaşmıştık. Bunun C++ da daha kapsamlı ve karmaşık olduğunu göreceğiz. Ancak, birkaç kavramı öğrenmek için öncelikle C ye geri dönmemiz gerekiyor.

C - type conversion

C ile kodlarken casting denilen şeyi hepimiz kullandık. Ancak C de iki tür cast olduğunu fark etmemiş olabilirsiniz. Değişken tipinin etrafında parantez olan syntaxı tanıyorsunuz. Buna "Explicit casting" (Açık tür dönüşümü) denir.

İkinci tür dönüşümü yöntemi ise "Implicit casting" (örtük tür dönüşümü) dir. Bu dönüşümü yapmak için özel bir syntax olmadığından, kullandığınızı fark etmemiş olabilirsiniz.

C de bazen bilmeden, bazen bilerek kullandığınız bu iki cast, her zaman aynı olmayan davranışlara sebep olur. Casting konusunun 2or kısmı da bu. Aynı syntaxin altında farklı davranışlar gösterilebiliyor. Bu davranışlardan biri olan conversion kavramı ile başlayacağız.

`int a = 42;` → reference value

`double b = a;` → Implicit conversion cast (örtük tür dönüşümü)

`double c = (double)a;` → Explicit conversion cast (açık tür dönüşümü)

double, 64 bitten oluşan bir float türüdür. Float-point sayılar ise, bilgisayarların küsüratlı sayıları bitler ile ifade etmesinin bir yöntemidir. Konuyu daha iyi anlamak için floatların hafızada nasıl temsil edildiğini araştırmanızı tavsiye ediyorum. SpanningTree youtube kanalının konu hakkındaki videosunu tavsiye ederim.

Asıl konuya dönecek olursak: Bir double değeri, aynı sayısal değere sahip olan bir integerdan bit düzeyinde farklıdır. Bu yüzden casting yapılırken, aynı sayısal değeri korumak için bit düzeyinde işlem yapılır.

Şu double b = a yaptığında arka planda şu dönüşüm yapılıyor:

a (integer 42) 32 bits

0000 0000.0000 0000.0000 0000.0010 1010



b (double 42) 64 bits

0.10000000100.010100000000000000000000000000-
0000 000000000000000000000000

Gördüğümüz gibi bu iki sayının değerleri aynı, ancak temsil edildikleri bit yapıları tamamen farklı.

Conversion yaparken, değişken tipleri arasındaki hiyerarşi önemli bir konudur. Double, integerdan da büyük ve daha hassas bir tip olduğu için, bir integerı sayının tam değerini kaybetmeden double'a dönüştürebiliriz.

Peki, tam tersini yapmak istesek ne olur? (double'ı integera dönüştürmek gibi) Bu, daha hassas (precise) bir değişken tipini daha az hassas bir tipe dönüştürmektir ve sayısal doğruluk kaybına sebep olabilir.

Örneğin değeri 10,125 olan bir double'ı integer'a cast'lersak yeni değer 10 olacaktır. 0,125 kadar doğruluk kaybımız var. Böyle bir dönüştürmeyi bir kere yaparsak problem olmayabilir, ancak bir döngü içinde sürekli hesaplamalar yaptığımızı, ve her hesabın sonucunun bir sonrakinin etkilediğini düşünün. Böyle durumlarda bu kayıplar çığ gibi büyüyerek kocaman bir hata oranı yaratabilir. Bu yüzden bu tür dönüşümleri (type demotion) bilerek ve etkilerini fark ederek yapmalıyız.

double d = a → Implicit promotion - problem yok. Veri kaybı olamaz.

int e = d → Implicit demotion - Tehlikeli! Implicit cast'leri yaptığımızı fark etmek zordur. Veri kaybı kolayca dikkatinizden kaçabilir.

int f = (int)d → Explicit demotion - Ne yaptığının farkındasın, sorun yok.

! Implicit demotion yaptığında uyarılmak için -Wconversion derleyici flagini kullan

C - type reinterpretation

Önceki konuda, C'nin değişken tipleri arasında dönüşüm yapabildiğini, ancak verinin yeni değişken tipine uyumlu olması için bitlerde değişiklik yaptığını görmüştük.

Şimdi ise daha ilginç bir dönüşüm türünü göreceğiz. Buda casting verinin kimliğine yapılır, ve bitlerde değişiklik olmaz. Yani aslında gerçek anlamda bir dönüşüm yoktur. Buna 'reinterpretation' (yeniden yorumlama) adı verilir.

Önceki konuda tipler arasında bir hiyerarşi olduğunu söylemiştik. Bazı değişken tipleri öbürlerinden daha hassastır. Adres tiplerinde de benzer bir hiyerarşi vardır. `void *`, her türlü adresi saklayabilmemizi sağlayan değişken tipidir. `void *`, diğer bütün adres türlerinden daha geneldir.

```
int main(void) {
```

```
float a = 420.042f;
```

```
void *b = &a → Implicit reinterpretation cast
```

```
void *c = (void *)&a → Explicit reinterpretation cast
```

```
void *d = &a → Implicit promotion -
```

Daha hassas bir adresi daha genel bir pointerda tutuyoruz. Sorun yok.

```
int *e = d → Implicit demotion - Tehlikeli!
```

Burada orijinalinde bir float'ın adresi olan bir adresi, integer pointerı olarak atadık.

Float ve integerların bit yapılarının farklı olduğunu biliyoruz. Yaptığımız bu şey sınır bozucu etkilere sebep olacak.

```
int *f = (int *)d → Bilinçli bir şekilde Explicit demotion
```

kullandım. Sıkıntı yok. Demotionları explicit yaparak, kendimizi dahil daha sonra kodu okuyacak herkese bunu bilerek yaptığımızı ve bunun bir programlama hatası olmadığını belli etmiş oluruz.


```

printf ("%p, %.f\n", &a, a);
printf ("%p\n", b);
printf ("%p\n", c);

printf ("%p\n", d);
printf ("%p, %d\n", e, *e);
printf ("%p, %d\n", f, *f);
}

```

Output:

```

0x7fff5247eaa4, 420.041992
0x7fff5247eaa4
0x7fff5247eaa4
0x7fff5247eaa4
0x7fff5247eaa4, 1137837408
0x7fff5247eaa4, 1137837408

```

→ identity cast (kimlik dönüşümü)
olduğu için adres aynı
ancak adres artık int olarak
okunduğu için değer değişti.
Bitler aynı, yorumlama şekli
farklı.

void *d bit gösterimi:

0100 0011 1101 0010 0000 0101 0110 0001

float *g = (float *)d;

int *h = (int *)d;

← aynı adres
(değişiklikle aynı bit düzeni)

*g = 420.041992

*h = 1137837408

olun

C - type qualifier reinterpretation

C'de type qualifier, "const" ve "volatile" gibi anahtar kelimelerdir. Daha önce "volatile" keywordünü kullanmaya pek fırsatımız olmadığı için, örneklerimizi const üzerinden vereceğiz.

```
int main(void) {
```

```
    int a = 42;
```

```
    int const *b = &a; → Implicit type qualifier cast
```

```
    int const *c = (int const *)&a; → Explicit type qualifier cast
```

Tabii ki: type qualifierların da hiyerarşisi vardır. Bu önceki gördüğümüz castlerden biraz farklı olmakla birlikte yine de doğru.

Bir const int'in adresi, bir int'in adresinden daha geneldir ve hiyerarşide daha yukarıdadır.

```
int const *d = &a; → Implicit promotion (Sorun yok)
```

```
int *e = d; → Implicit demotion (COMPILE OLMAZ.)
```

Önceki castlerde, Implicit demotion yapmanın sorunlar yaratacağını, tespit etmesi zor buglara sebep olacağını konuşmuştuk. Buradaki sorun biraz daha farklı.

Const bir adresi, const olmayan bir adres yapmaya çalışırsak kod derlenmeyecektir.

```
int *f = (int *)d; → Explicit demotion (Compile olur)
```

```
}
```


Bir adresin type qualifierını deęiřtirmek pek tavsiye edilmez.
Eđer kendinizi bu castingi yapmak durumunda bulduysanız iki ihtimal var.

Ya kodunuzda bir hata yaptınız - ki bu durumda cast yapmak yerine programı yeniden tasarlamalısınız.-

Ya da kendinize ait olmayan bir kodu kullanıyorsunuzdur. (bir kütüphane gibi).
Şu senaryoyu düşünelim:

Siz, iyi kodladığınız için programınızda deęişmesini istemediğiniz bir deęişkenin adresini const yaptınız.

Sonra, bir kütüphaneden bir fonksiyon kullandınız. Bu fonksiyon da deęişkende bir deęişiklik yapmıyor (yani const parametre için yazılmış) ancak kütüphaneyi geliřtiren kiři fonksiyon prototipinde bu parametre için const kullanmayı unuttu. Eğer fonksiyonun deęişken üzerinde deęişiklik yapmayacağına gerçekten eminsek, bu durumda bu fonksiyonu kullanabilmek için type qualifier cast yapmayı düşünebiliriz. Kütüphanenin hatalı kodlanmış olmasından biz sorumlu olmayız. Tabii ki bu gibi durumlar dışında hata sizdedir ve kendi kodunuzu düzeltmeniz gerekir.

Ve bu konuyla birlikte Cde cast konusunun sonuna geldik. Artık, bu konuların C++ da uygulaması konusuna geçebiliriz.

Upcast and Downcast

C++ da, class kavramının varlığından ortaya çıkan bir casting hiyerarşisi göreceğiz. Cde casting hakkında konuşurken daha genel (hiyerarşide yukarıda) ve daha özel (hiyerarşide aşağıda) pointer ve değişken tipleri görmüştük. Classlar için de benzer bir durum var.

class Parent { }; → parent class. Daha genel, hiyerarşide daha yukarıda.

class Child1: public Parent { };
class Child2: public Parent { }; > Child classlar. Daha özelleşmiş, hiyerarşide daha aşağıda

Inheritance kavramı sayesinde, doğal olarak parenttan child'a doğru düşen bir hiyerarşi vardır. Önceki projelerden, bir parent class ve child class pointerinin birbirine uyumlu olduğunu biliyoruz. Class adreslerinin casting davranışlarını görmek için bu uyumluluğu kullanacağız.

```
int main(void) {
```

```
    Child1 a;
```

```
    Parent * b = &a → Implicit "reinterpretation" cast. Her "Child1" aynı zamanda bir "Parent" olduğu için, bir "Parent *" pointeri, bir Child1'in adresini tutabilir. Bu, bir float adresini bir void adresine dönüşüm yaptığımız örneğe benziyor. (Özelden genele, yani promotion yapıyoruz.)
```

```
    Parent * c = (Parent *) &a → Explicit "reinterpretation" cast
```

```
    Parent * d = &a → Implicit upcast (Sorun yok)
```

```
    Child1 * e = d → Implicit downcast (COMPILE OLMAZ)
```

```
    Child2 * f = (Child2) d → Explicit downcast (Compiler'a explicit olarak söylediğimiz için compile olur.)
```

↓

Yaptığımız bu cast compile olacaktır, ancak runtime'da problemlere sebep olacaktır. Bildiğimiz gibi, "d" bir Child1 objesinin adresini tutuyor. Child1 ve Child2 classlarının implementasyonu birbirinden farklı olabileceğinden, bu runtime sırasında dramatik etkilere sebep olacaktır. Bir Child1'i, Child2 olarak yorumlamaya çalışıyor. Bu konuyu iyi açıklayabilmek için Cnin casting syntaxini kullandık. Ancak ileriki konularda göreceğiz ki C++, bize çeşitli yeni castler ve her durum için spesifik castler seçmemize olanak sağlayacak bir syntax sunacaktır. Böylece, bunun gibi runtime problemleriyle karşılaşmadan derlemeyi durdurabileceğiz.

Static Cast

```
int main(void) {
```

```
    int a = 42
```

```
    double b = a → Implicit promotion (Sorun yok)
```

```
    int c = b → Implicit demotion (Olmez!)
```

```
    int d = static_cast<int>b; → Explicit demotion (Tamam)
```

↓ ↓ ↓
keyword hangi tipe dönüştürülecek
 dönüştüreceğim ifade

// C++da, static cast syntaxı yukarıdaki gibidir. Bu castı önceden şöyle yapıyorduk:

// int d = (int)b → Artık Cde olmadığımızı göre, bu syntaxı kullanmayı bırakacağız. Bundan sonra, basit değerlerde dönüşüm yapmak için C++ya ait olan static casti kullanacağız.

Static Cast - Upcast and Downcast

```
class Parent {};  
class Child1 : public Parent {};  
class Child2 : public Parent {};
```

```
class Unrelated {};
```

```
int main(void) {
```

```
    Child1 a;
```

```
    Parent *b = &a → Implicit upcast (Sorun yok)
```

```
    Child c = b → Implicit downcast (Olmaz)
```

```
    Child2 *d = static_cast<Child2*>(b) → Explicit downcast  
                                           (Compile olur)
```

↓

Bu dönüştürme teknik olarak yapılabilir ancak sorunlar yaratacaktır.

(Dikkat edin, aslında bir Child1 olan objeye Child2 pointerı veriyor.

Böyle bir şeyi daha önce gördük.) C++'da, bu gibi davranışları tespit edip önlemimizi sağlayacak bir cast var. İleriki konuda göreceğiz.

```
    Unrelated *e = static_cast<Unrelated*>(&a);
```

↓

Explicit conversion. Bu satır derlenmeyecektir. Static cast'in bir avantajı, aynı inheritance ağacında olmayan tiplerin birbirine dönüşümünü engellemesidir. &a, bir Child1'in adresidir. Aynı miras ağacında olmayan Unrelated class adresine dönüştürülemez.

```
}
```


Dynamic Cast

Dynamic cast, C++ castleri arasında en farklı olanıdır. En önemli farkı, dönüşümün compilationda (derlenirken) değil, runtime'da (program çalışırken) gerçekleşmesidir.

Bu da demektir ki, program derlenmekte sorun yaşamayıp, dynamic cast programı çalıştırdığınız sırada başarısız olabilir. Yani, programınız içerisinde dynamic cast hatalarıyla başa çıkacak mimariyi siz kodlamalısınız.

Bu castin, ilkiyle bağlantılı olan ikinci özelliği ise yalnızca polymorphic instance ile çalışmasıdır. Yani, subtype polymorphism ile çalışacaktır. En az bir member fonksiyon virtual olmalıdır. Subtype polymorphism, RTTI (runtime type info) denilen şeyi aktive eder. Bu, class'te depolanan ancak bizim görmediğimiz bir sürü faydalı bilgiyi içerir ve dynamic castimizin mümkün olup olmadığını kontrol edebilmemizi sağlar. Dynamic casti, yalnızca downcast için kullanacağız. Bu cast, sadece pointer ve referanslar ile çalışır.

```
class Parent { public: virtual ~Parent(void) {} };  
class Child1: public Parent {};  
class Child2: public Parent {};
```

```
int main(void) {
```

```
    // Dynamic cast - pointer
```

```
    Child1 a;
```

```
    Parent* b = &a; → Implicit upcast (sorun yok)
```

```
    Child1* c = dynamic_cast<Child1*>(b) → dynamic cast,  
                                           dönüştürülmüş adresi  
                                           ya da castin imkansız  
                                           olması durumunda NULL  
                                           pointer döndürür.
```

```
    if (c == NULL) {  
        std::cout << "Conversion is NOT ok" << std::endl;  
    } else {  
        std::cout << "Conversion is ok" << std::endl;  
    }  
}
```

Dynamic cast'ten dönen pointerın NULL olup olmadığını kontrol ederek hata kontrolü yapılabilir.

Bir Parent adresini, Child1 adresine dönüştürmek burada sorun yaratmayacaktır. Emin olmak için, runtime çıktısına bakacağız.

// Dynamic cast - Reference

Dynamic cast, referans ile yapıldığında biraz farklıdır. Örnekte NULL olup olmadığını kontrol edebileceğimiz bir pointer olmadığı için hata kontrolünde exception kullanmalıyız. C++, bize bu durumda kullanabilmemiz için std::bad_cast exceptionunu veriyor.

```
try {
```

```
    Child2 &d = dynamic_cast<Child2 &>(*b);
```

```
    std::cout << "Conversion is ok" << std::endl;
```

```
}
```

```
catch (std::bad_cast &bc) {
```

```
    std::cout << "Conversion is NOT ok : " << bc.what() << std::endl;
```

```
    return 0;
```

```
}
```

Eğer dikkat ederseniz, burada bir Child1 referansını Child2'ye castlemeye çalışıyor. Bu durumun problematik olduğundan daha önce bahsetmiştik. Dynamic cast, bunu algılayacak ve exception fırlatacaktır.

```
}
```

Dynamic cast, program çalışırken gerçekleştiği için hatalı olsa bile programımızı başarılı şekilde derlenecektir. Bu hataları programın içinde yakalamak bizim elimizdedir.

Output:

Conversion is ok

Conversion is NOT ok: std::bad_cast

Dynamic casti ne durumlarda kullanmamız gerektiğini merak ediyor olabilirsiniz.

Şu ana kadarki her örnekte, yerine static cast kullanabilirdik. (Nasıl olsa static cast güvenli bir şekilde kullanmayı öğrendik.)

Ancak, programlamada henüz pek hisar neşir olmadığımız bir konu var: uygulamalara plugin eklemek. Bunu yaparken, pluginimizde dynamic_cast yerine static_cast kullandığımız durumlarda ana uygulama çökebilir.

Bu konuyu ben de derinlemesine bilmediğim için kendi araştırmanızı yapmanızı tavsiye ediyorum.

Reinterpret Cast

Reinterpret cast, adı üstünde yeniden yorumlama yapmamıza olanak tanır. Ayrıca bu cast ile upcast ve downcast de yapabiliriz. Bu, C++ göreceğimiz en serbest cast olacak. Herhangi bir adresi, herhangi bir tür başka adres şeklinde yorumlayabileceğiz. (Tabii ki, bunun yan etkilerini göz önünde bulundurarak.) Bu casti kullandığımız basit bir örneğe bakalım.

```
int main(void) {  
    float a = 420.042f;  
    void *b = &a; → Implicit promotion, (sorun yok)  
    int *c = reinterpret_cast<int*>(b); → Implicit demotion (tamam)  
    int &d = reinterpret_cast<int&>(b); → Implicit demotion (tamam)  
}
```

reinterpret cast yapıırken hiçbir semantik kontrolü yapılmaz. Bir diğer deyişle, compiler size tamamen güvenir. Herhangi bir adresi, herhangi bir adres tipi olarak yeniden yorumlar, ve bunun sonuçlarından siz sorumlu olursunuz.

Bu örnekte göstermedik, ancak bu dönüşümü rastgele classlar ile de yapabilirsiniz. Derleyici sizi durdurmayacaktır. Ancak runtime'da sorunlarla karşılaşabilirsiniz.

Tabii ki reinterpret cast'in var olmasının iyi bir sebebi var. Adresleri yeniden yorumlamak kullanışlı olabilir. Mesela, bir network'ten veri aldınız. Bu rastgele veriler size genellikle void* olarak gelecek ve bu verileri anlamlandırmak için yeniden yorumlama yapmanız gereken durumlar olacak. Bu, reinterpret cast'in yaygın bir kullanım alanıdır.

Reinterpret cast, her şeyi her şeye dönüştürebildiği için ne istediğinizi bilseniz iyi edersiniz. Çünkü bir hata yaparsanız dramatik sonuçlarla karşılaşabilirsiniz.

Const Cast

Önceden gördüğümüz C++ castlerinin hiçbiri, type qualifier'ları üzerinde dönüştürme yapamıyordu. Const cast, bu işlemi yapmamıza yarayacak bir cast'tir. Nasıl çalıştığını görmek için bir örneğe bakalım.

```
int main(void) {
```

```
    int a = 42;
```

```
    int const *b = &a; → Implicit promotion (sorun yok)
```

// C cast'te gördüğümüz type qualifier cast konusunun aynısı burada da var.

Daha özel bir veri tipini, daha genel bir veri tipine dönüştürerek promotion yaptık. generic/accurate (genel/özel) tip kavramlarının "const" kavramını pek iyi açıklamadığını söylemem gerek. Ancak yine de, const bir veri tipinin, değiştirilebilir bir veri tipine göre hiyerarside daha üstte olduğunu söyleyebiliriz.

Değiştirilebilir (mutable) bir veriyi sabit (const) bir veriye dönüştürdüğümüzde sorun çıkmazken, const'un çalışma prensibi yüzünden tam tersi mümkün değildir.

```
    int *c = b; → Implicit demotion (COMPILE OLMAZ)
```

Yine de yapmak istiyorsam, const cast kullanmalıyım:

```
    int *d = const_cast<int*>(b); → Explicit demotion (tamam?)  
}
```

Const cast yaparken dikkat etmemiz gereken noktalar C'de type qualifier cast yaparken dikkat etmemiz gereken konularla aynıdır. Eğer const_cast kullanmamız gerektiyse, bu baya kötü bir belirtidir ve genellikle kod tasarımınızın yanlış yapılmış olduğunu gösterir. Bu durumda const_cast kullanmamak için kodunuzu yeniden tasarlamalısınız. Bu kurala uyarak olabildiğince fazla const kullanırsak (değişmemesi gereken tüm değerler), programımız bakıma uygun ve daha stabil olacaktır.

Eğer const_cast yapmak zorunda kalırsanız yapın, ancak bunu yapmak için iyi bir sebebiniz olduğundan emin olun. C- type qualifier reinterpretation başlığında geri dönerek oradaki örneği hatırlayabilirsiniz.

Const cast, göreceğimiz son C++ cast'iydi. Sonraki başlıklarda, cast ile alakalı birkaç ek konuya değineceğiz.

Cast Operators

C++da type cast operatörleri, Implicit (örtük) dönüşümler yapabilmek için classımız içerisinde spesifik operatörler tanımlayabilmemizi sağlar. Böylece classımızı istediğimiz bir tipe dönüştürebiliriz. Bu konuyu anlamamanın en iyi yolu bir örnek incelemek.

```
class Foo {
```

```
public:
```

```
    Foo(float const v) : _v(v) {} → Constructor
```

```
    float getV(void) { return this->_v; } → getter
```

```
    operator float() { return this->_v; }
```

```
    operator int() { return static_cast<int>(this->_v); }
```

↓

! cast operatörleri.

```
private:
```

```
    float _v;
```

```
};
```

```
operator    int    { return static_cast<int>(this->_v); }
```

↓
keyword

↓
hangi tipe
castlediğim

↓
Classım, belirtilen tipe (burada int)
dönüştürüldüğünde döndürülecek ifade.


```
int main(void) {
```

```
    Foo a(420.024f);
```

```
    float b = a;    → Tanımladığımız cast operatörü sayesinde,  
                     a class örneğimizi direkt olarak kullanarak  
                     bir b floatı initialize edebildik.  
                     Derleyici, class tanımında float için bir cast  
                     operatörü olup olmadığını kontrol eder ve bulursa  
                     tanımladığımız aksiyon gerçekleşir.  
                     (return this → -v)
```

```
    int c = a;    → int için de bir cast operatörü tanımlamıştık.  
                  Yine class içerisindeki tanıma uyarak dönüştürme  
                  işlemi gerçekleşecektir.  
                  (return static_cast<int>(this → -v))
```

```
    std::cout << a.getV() << std::endl;  
    std::cout << b << std::endl;  
    std::cout << c << std::endl;  
}
```

Cast operatörleri, aritmetik tiplere benzer classlar yazmak istediğimizde son derece kullanışlıdır.

Output:

420.024
420.024
420

Cast operatörleri bu kadar. Çok pratik, ancak her zaman ihtiyacımız olmayacak bir şey. Ancak bazen class'ınızı kullanması daha doğal hale getirmenin iyi bir yoludur.

Explicit

Explicit, bir C++ keywordüdür. Bunu anlamak için örneği inceleyelim.

```
class A {};  
class B {};
```

```
class C {
```

```
public:
```

```
    C(A const&_) {return;} → Constructor, Parametre olarak A alır.  
    explicit C(B const&_) {return;} → Constructor, Parametre olarak  
                                     B alır.
```

```
};
```

Yukarıda gördüğümüz gibi C classının iki farklı constructorı var. Biri, A classını parametre olarak alıyor, öbürü ise B'yi alıyor ve explicit keywordünü kullanmış. İlk constructorun davranışına odaklanarak bunlar arasındaki farkı açıklayacağız.

Aşağıda parametre olarak C referansı alan bir fonksiyonum var.

```
void f(C const&_) {  
    return;  
}
```

Diyelim ki, bu fonksiyonu şu şekilde çağırdım:

```
f(A());
```

İyi de, ben bu fonksiyona A örneği attım, benden C istiyor. Çalışır mı ki? Çalışır. C++, sistematik Implicit cast'e izin verir. Eğer f fonksiyonumu çağırır ve parametre olarak ona bir A class örneği verirsem, Derleyici bir Implicit cast yapmaya çalışacaktır. Class tanımına bakacak, A örneği alıp C örneği yapan constructoru görecektir ve $A \rightarrow C$ implicit castini yaparak fonksiyonun çalışmasını sağlayacaktır. (Implicit construction)

Peki ya $f(B());$ şeklindeyse? Çalışmaz. "explicit" keywordü, derleyicinin $B \rightarrow C$ Implicit castini yapmasını engelliyor. Eğer B alan constructoru kullanıp f çağırarak istiyorsak, önce açık olarak bu constructor ile bir C örneği oluşturup o örneği parametre olarak vermeliyiz.

Conclusion

Bu konu uzun olduğu için, castler ve özellikleri hakkında referans olarak kullanabileceğiniz bir tablo ekledim.

Cast	Conversion (dönüşüm)	Reinterpretation (yeniden yorumlama)	Upcast	Downcast	Type qualifier c.
Implicit	✓	-	✓	-	-
static-cast	✓	-	✓	✓	-
dynamic-cast	-	-	✓	✓	-
const-cast	-	-	-	-	✓
reinterpret-cast	-	✓	✓	✓	-
Legacy C cast	✓	✓	✓	✓	✓

Cast	Semantic check (semantik kontrolü)	Reliable at run (runtime'da güvenilir)	Tested at run (runtime'da test edilir)
Implicit	✓	✓	-
static-cast	✓	-	-
dynamic-cast	✓	✓	✓
const-cast	-	-	-
reinterpret-cast	-	-	-
Legacy C cast	-	-	-

! Hatırlatma : Legacy C casti C++ projelerinde kullanmayacağız.